

AI Disclosure

In this project the following AI models were used:

- Claude Sonnet 5
- Claude Pro (Fable, Opus, Sonnet, Code)
- Google Gemini Flash 3.5
- Google Notebook LM
- GitHub Copilot

Unless stated otherwise, all used models were used in the free version

Prompt examples

To demonstrate the prompting techniques used two brief examples can be found on the next pages.

1. Definition of the task and development of a BO loop design with Gemini (Pages 2 - 12)
2. Iterative bug fixing and improvement of code with Claude (Pages 13 – 15)

Group 8

[Hier eingeben]

Dziyana Medvetskaya (243210)

Julian Krüger (235696)

Philipp Johanning (232627)

Nils Guhl (176506)

1. Chat (gemini.google.com/ Modle 3.1 (Pro))

User:

Guten Morgen,

ich arbeite momentan an einer Projektabgabe für meinen Master Universitätskurs "Machine learning for engineers". Die Aufgabenstellung ist hier angehängt. Grundsätzlich geht es um das Design eines Bayesian Optimisation loop mit dem in möglichst wenigen Versuchen bzw. möglichst günstig ein Rezept für den Pilot scale definiert wird mit dem der höchste Yield erzielt wird. Einflussparameter sind T pH F1 F2 F3. T und pH scheinen Core-Parameter zu sein. Wenn diese Core Parameter falsch gewählt sind kann auch F1 bis F3 nichts an einem schlechten Ergebnis ändern. Zusätzlich wird angenommen, dass das T und pH Optimum unabhängig von allen anderen Größen ist.

Mein Optimierungsansatz soll folgender Struktur folgen:

1. Optimierung von T: Hier soll das optimale T festgelegt werden. Da das Optimum recht schmal ist und die anderen Parameter noch unoptimiert sind, ist es hier in der Vergangenheit Probleme gegeben, da das Optimum sich nicht ganz so klar vom Rauschen abgesetzt hat.
2. Optimierung von pH: Hier soll das vorher gefundene T Optimum festgesetzt werden und anschließend das pH Optimum gefunden werden. Hier sollte sich durch das festgesetzte T bereits ein deutlicheres Optimum ausbilden.
3. Optimierung von F1 bis F3: Hier soll bei festgesetzten optimalen T und pH die drei Feedrates optimiert werden.

Wie bereits erwähnt gibt es drei Versuchs scales: micro (10 Euro), bench (200 Euro), pilot (2000 Euro). Die durchschnittlichen auf die Messwerte normierten Standardabweichungen liegen bei: micro: 0.1225, bench: 0.016, pilot: 0.0028.

Bitte mache mir Vorschläge mit welchen Methoden ich die BO-Loops designen könnte, basierend auf den Voraussetzungen der drei Schritte. Mache Vorschläge zu Initialisierungsmethode, Acquisition function, Surrogate Model und falls notwendig weiteren Designelementen.

Mache nicht anderes. Keine Konfabulation.

Bevor du mit der Aufgabe beginnst, fasse meinen Prompt bitte in 5 Sätzen zusammen um klar zumachen, was du verstanden hast.

Google Gemini:

Hier ist zunächst die gewünschte Zusammenfassung deines Prompts:

Du bearbeitest ein Masterprojekt für den Kurs „Machine Learning Methods for Engineers“, in dem du einen Bayes'schen Optimierungs-Loop (BO) für einen simulierten Bioreaktor entwerfen sollst. Das übergeordnete Ziel ist es, die Ausbeute auf der Pilot-Skala zu maximieren und dabei die Gesamtkosten über die drei Skalen (Micro, Bench, Pilot) zu minimieren. Unter der Annahme, dass die Kernparameter Temperatur (T) und pH-Wert unabhängig von den Feedraten sind, planst du einen sequenziellen Optimierungsansatz. Dieser Ansatz sieht vor, zunächst die Temperatur, dann den pH-Wert und abschließend die drei Feedraten schrittweise zu optimieren und die gefundenen Optima jeweils für die nachfolgenden Schritte festzusetzen. Um diesen Plan umzusetzen, benötigst du nun konkrete Vorschläge für das Design der BO-Loops, insbesondere bezüglich Initialisierung, Surrogate-Modellen und Akquisitionsfunktionen für die drei definierten Teilaufgaben.

Dein Ansatz, das Problem durch die Annahme von Unabhängigkeit in kleinere Teilprobleme ($1D \rightarrow 1D \rightarrow 3D$) zu zerlegen, ist eine pragmatische Methode, um den Suchraum zu verkleinern und Kosten zu sparen. Da die Aufgabenstellung jedoch explizit fordert, in jeder Iteration sowohl das nächste Rezept als auch die nächste Skala zu wählen, muss die Multi-Fidelity-Natur (Micro, Bench, Pilot) der Datenquellen in jedem deiner drei Schritte abgebildet werden.

Hier sind konkrete Designvorschläge für deine drei Optimierungsschritte:

Generelles Designelement (Für alle Schritte gültig)

- **Surrogate Model:** Ein **Multi-Fidelity Gaussian Process (MF-GP)** (z. B. Co-Kriging oder ein Continuous-Fidelity Modell). Da du Daten aus drei Skalen mit stark unterschiedlichem Rauschen und systematischem Bias fusionieren musst, reicht ein Standard-GP nicht aus. Ein MF-GP lernt die Korrelation zwischen der stark verrauschten, aber günstigen Micro-Skala und der sauberen, teuren Pilot-Skala.

- **Cost-Awareness:** Da die Kosten pro Experiment variieren (10 Euro, 200 Euro, 2000 Euro), sollte deine Akquisitionsfunktion in jedem Schritt kostenbewusst sein, z. B. durch Division der Metrik durch die Kosten der jeweiligen Skala s :

$$\alpha_{\text{cost}}(\mathbf{r}, s) = \frac{\alpha(\mathbf{r})}{\text{Cost}(s)}.$$

Schritt 1: Optimierung von T (1D-Suchraum)

Das Ziel ist es, trotz des hohen Rauschens und des schmalen Optimums den wahren Peak zu identifizieren.

- **Initialisierungsmethode:** Verwende **Latin Hypercube Sampling (LHS)** oder **Sobol-Sequenzen**, um den Bereich für T (20 – 60°C) gleichmäßig abzudecken. Führe ca. 5-10 Messungen auf der *Micro-Skala* und 1-2 Messungen auf der *Bench-Skala* durch, um dem MF-GP direkt ein Gefühl für den Bias zwischen den Skalen zu geben.
- **Surrogate Model Spezifikation:** Setze das Beobachtungsrauschen für die Micro-Skala im GP explizit auf die bekannte Varianz (Standardabweichung $\sigma = 0.1225$, also $\sigma^2 \approx 0.015$). Verwende einen Matérn-5/2-Kernel, da dieser sich gut für physikalisch/chemische Prozesse eignet, die nicht unendlich glatt (wie bei RBF) sind.
- **Acquisition Function: Upper Confidence Bound (UCB).**
 - *Warum:* Da das Optimum schmal ist und das Rauschen hoch, besteht die Gefahr, dass Expected Improvement (EI) sich zu früh auf Rauschartefakte einschießt (Exploitation). Mit UCB kannst du den Parameter β so steuern, dass gezielt Regionen mit hoher Unsicherheit (Exploration) abgetastet werden.

Schritt 2: Optimierung von pH (1D-Suchraum, bei fixiertem T)

Hier ist das Signal-Rausch-Verhältnis laut deiner Annahme bereits besser, da T im Optimum liegt.

- **Initialisierungsmethode:** LHS über den pH-Bereich (3 – 9.5). Da die Temperatur bereits fix ist, kannst du hier etwas aggressiver auf die Bench-Skala setzen (z. B. 3 Bench-Messungen, 5 Micro-Messungen), da das Optimum klarer hervortreten sollte.
- **Surrogate Model Spezifikation:** Der gleiche MF-GP, jedoch wird hier nur noch die pH-Dimension variiert. Der GP kann nun mit einer geringeren Längenskala (Length-Scale) im Kernel rechnen, falls der pH-Einfluss sprunghafter ist.
- **Acquisition Function: Expected Improvement (EI).**

- *Warum:* Da das Rauschen durch das feste Temperatur-Optimum weniger kritisch ins Gewicht fällt, ist EI hier effizienter als UCB. EI fokussiert sich natürlich auf die Ausbeute (Yield Y) und balanciert Exploration und Exploitation ohne manuelles Tuning eines β -Parameters.

Schritt 3: Optimierung von F_1 bis F_3 (3D-Suchraum, bei fixiertem T und pH)

Hier erfolgt das Feintuning der Feedraten, das maßgeblich die Performance auf der Pilot-Skala entscheidet.

- **Initialisierungsmethode:** LHS im 3D-Raum für die Parameter F_1, F_2, F_3 (jeweils im Bereich 0 – 2 g/L/h). Hier solltest du mindestens eine Messung auf der *Pilot-Skala* einbauen, damit der MF-GP den finalen Bias-Shift zur Produktionsumgebung erlernen kann.
- **Surrogate Model Spezifikation:** Behalte den MF-GP bei, nutze hier aber idealerweise Automatic Relevance Determination (ARD) für die Kernel, um festzustellen, ob eine der Feedraten einen dominanteren Einfluss hat als die anderen.
- **Acquisition Function: Cost-aware Expected Improvement oder Information-theoretic approaches (wie Predictive Entropy Search).**
 - *Warum:* Hier wird das Budget am stärksten strapaziert. Die Akquisitionsfunktion muss streng bewerten, ob das Ausführen eines Bench-Experiments (200 Euro) genug Information liefert, um das 2000-Euro-Pilot-Experiment unnötig zu machen, oder ob direkt ein Pilot-Run zur Bestätigung des globalen Optimums fällig ist.

Ein kritischer Hinweis zur Architektur:

Dein sequenzieller Ansatz birgt das Risiko, dass du auf einem lokalen Optimum hängen bleibst, falls die Annahme der vollständigen Unabhängigkeit zwischen (T , pH) und (F_1, F_2, F_3) in der Simulation nicht zu 100 % zutrifft. Die meisten BO-Ansätze lösen das 5D-Problem simultan, nutzen aber die Micro-Skala extrem intensiv (hunderte Runs für wenig Geld), um den 5D-Suchraum initial abzugrasen, bevor sie die teuren Skalen ansteuern.

Wie planst du, den systematischen Bias der günstigeren Skalen (Micro, Bench) zu quantifizieren und mathematisch in deinem Surrogate-Modell zu korrigieren, bevor diese Daten zur Vorhersage des Pilot-Ergebnisses genutzt werden?

User:

Aufgrund des in dem in der Aufgabe künstlich sehr niedrigem Budget (auf jeden Fall kleiner als 15000 Euro) müssen die Versuche geldkritisch bewertet werden. Wenn jedoch ein teurer Versuch notwendig ist kann dieser natürlich durchgeführt werden. Zum Schluss muss jedoch auf jeden Fall ein Pilotversuch als Benchmark durchgeführt werden, dessen Kosten auch in das Ergebnis eingehen.

Zur Quantifizierung des Kosten Bias hatte ich daran gedacht die erwartete Verbesserung an einem Punkt mit den Kosten für ein Scale und der Standardabweichung des Scales aufzuwiegen.

Google Gemini:

Deine Überlegung trifft den Kern des Multi-Fidelity-Problems exakt. Da der zu übertreffende Benchmark für die mittleren Gesamtkosten bei 15000 Euro liegt, ist ein extrem kostenbewusstes Vorgehen zwingend erforderlich. Auch die strategische Einplanung des finalen Pilot-Versuchs ist essenziell, da die finale Auswertung explizit den besten gefundenen Produktwert auf der Pilot-Skala (Y_{pilot}) fordert.

Dein Ansatz, die erwartete Verbesserung (Expected Improvement, EI) gegen die Kosten und die Standardabweichung (das Rauschen) abzuwägen, ist der absolute Standard in der modernen Bayes'schen Optimierung und wird oft als **Cost-aware Expected Improvement** oder "Expected Improvement per Penny" bezeichnet.

Hier ist die strukturelle und mathematische Einordnung deiner Idee für die praktische Umsetzung:

1. Das Budget-Management

Da du für den finalen Nachweis zwingend einen Pilot-Run benötigst, musst du ein **hartes Budget-Limit** implementieren.

- Wenn dein Zielbudget beispielsweise 14000 Euro beträgt, setzt du das Limit für den aktiven Optimierungs-Loop auf 12000 Euro.
- Sobald dieses Limit erreicht ist, stoppt der Loop und das verbleibende Geld wird genutzt, um das vom Surrogate Model vorhergesagte globale Optimum einmalig auf der Pilot-Skala zu validieren.

2. Die Formalisierung deiner Acquisition Function

Deine Intuition, Kosten und Rauschen zu verrechnen, lässt sich mathematisch elegant lösen. Anstatt die Standardabweichung manuell in die Akquisitionsfunktion zu

programmieren, teilt man die Arbeit zwischen dem Gaussian Process (GP) und der Akquisitionsfunktion auf.

Die Rolle des Surrogate Models (GP): Der GP übernimmt das Rauschen und den Bias. Wenn du eine stark verrauschte Microplate-Skala abfragst, die am günstigsten, aber am stärksten verzerrt und verrauscht ist, spiegelt sich diese hohe Standardabweichung in einer hohen Vorhersageunsicherheit (Varianz) des GPs wider. Ein gut trainierter Multi-Fidelity GP reduziert automatisch die erwartete Verbesserung für stark verrauschte oder unkorrelierte Skalen.

Die Formulierung der Akquisitionsfunktion:

Die Akquisitionsfunktion α für einen Rezept-Vektor $\mathbf{r}$ und eine Skala s berechnet sich dann rein über das Verhältnis von Informationsgewinn (evaluiert für die Pilot-Skala) zu Kosten der jeweiligen Skala:

$$\alpha_{\text{cost-aware}}(\mathbf{r}, s) = \frac{\text{El}_{\text{pilot}}(\mathbf{r} \mid \text{Daten von Skala } s)}{\text{Cost}(s)}$$

- El_{pilot} ist die erwartete Verbesserung des Produktwertes auf der *Pilot-Skala*.
- $\text{Cost}(s)$ sind die Kosten der Skala s (also 10, 200 oder 2000 Euro).

Dieser Bruch sorgt dafür, dass ein Bench-Versuch nur dann gewählt wird, wenn er pro investiertem Euro mehr Wissen über das Pilot-Optimum liefert als 20 Micro-Versuche. Ein teurer Pilot-Versuch, der am teuersten und am wenigsten verrauscht ist, wird vom Algorithmus nur dann selbstständig gewählt, wenn die Unsicherheit in einer extrem vielversprechenden Region so hoch ist, dass die günstigeren Skalen aufgrund ihres Bias keine verlässlichen Informationen mehr liefern können.

3. Erweiterung: Informations-theoretische Ansätze

Falls dir Expected Improvement in der Praxis zu stark auf lokale Optima fokussiert ist, kannst du deine Idee auch in Richtung der Informationsgewinnung verschieben.

Methoden wie **Predictive Entropy Search (PES)** bewerten nicht die direkte Verbesserung des Yields, sondern die Reduktion der Unsicherheit über die Position des globalen Optimums:

$$\alpha_{\text{PES-cost}}(\mathbf{r}, s) = \frac{H(x^*) - \mathbb{E}[H(x^* \mid Y_s(\mathbf{r}))]}{\text{Cost}(s)}$$

Hierbei ist $H(x^*)$ die Entropie (Unsicherheit) über den wahren optimalen Rezept-Vektor. Diese Methode ist oft robuster gegenüber hohem Rauschen, mathematisch und rechnerisch jedoch deutlich aufwendiger zu implementieren als EI.

Welche Python-Bibliotheken (z. B. BoTorch, GPyOpt oder scikit-optimize) ziehst du für die Implementierung in Betracht, um den Multi-Fidelity Gaussian Process aufzubauen?

User:

Ich kenne mich nicht gut mit den Python-Bibliotheken aus. Bitte erkläre mir die für das hier beschriebene Problem die relevantesten und gib eine Empfehlung ab an welcher Stelle ich welche Nutzen sollte.

Keine Konfabulation.

Google Gemini:

Für die Umsetzung deines Multi-Fidelity- und kostenbewussten Optimierungsproblems kommen Standard-Bibliotheken wie *scikit-learn* schnell an ihre Grenzen, da sie die Verrechnung unterschiedlicher Datenqualitäten und Kosten nicht nativ unterstützen.

Hier sind die drei relevantesten Python-Bibliotheken für dein Projekt, gefolgt von einer konkreten Einsatzempfehlung.

Die relevantesten Bibliotheken im Überblick

1. BoTorch (Die State-of-the-Art Empfehlung)

BoTorch basiert auf PyTorch und ist aktuell die modernste und flexibelste Bibliothek für Bayes'sche Optimierung. Sie wird von Meta gepflegt und ist genau für komplexe, maßgeschneiderte BO-Schleifen gebaut.

- **Vorteile:** Sie besitzt native, hochentwickelte Unterstützung für **Multi-Fidelity-Modelle** (über MultiFidelityGP) und **kostenbewusste Akquisitionsfunktionen** (wie qMultiFidelityExpectedImprovement). Du kannst exakt definieren, wie das Rauschen der Micro-Skala im Vergleich zur Pilot-Skala gewichtet wird.
- **Nachteile:** Höhere Lernkurve, da man grundlegende PyTorch-Kenntnisse (Tensoren) benötigt.

2. Emukit (Die einsteigerfreundliche Alternative)

Emukit ist ein Toolkit, das speziell für das Design von Experimenten entwickelt wurde und auf älteren mathematischen Bibliotheken (wie GPy) aufbaut. Es wurde explizit für

Anwendungsfälle in der Industrie und im Ingenieurwesen entwickelt, bei denen Simulationen unterschiedlicher Genauigkeit (Multi-Fidelity) kombiniert werden müssen.

- **Vorteile:** Extrem intuitive API für Multi-Fidelity-Probleme. Es hat eingebaute Klassen für lineare Multi-Fidelity-GPs und fertige Schleifen für kostenbewusste Optimierung.
- **Nachteile:** Wird von der Community nicht mehr ganz so aktiv weiterentwickelt wie BoTorch.

3. Scikit-Optimize (skopt) (Der einfache Standard)

Diese Bibliothek ist sehr bekannt und extrem einfach zu bedienen, wenn man bereits mit *scikit-learn* gearbeitet hat.

- **Vorteile:** Minimaler Codeaufwand für Standard-Probleme.
- **Nachteile: Nicht geeignet für dein Hauptproblem.** Sie unterstützt von Haus aus kein echtes Multi-Fidelity-Lernen. Du könntest sie nur nutzen, wenn du den Multi-Fidelity-Ansatz komplett verwirfst und jede Skala isoliert optimierst (was deine Kosten in die Höhe treiben würde).

Konkrete Empfehlung für dein Projekt

Da es sich um einen universitären Masterkurs im Bereich Machine Learning handelt, solltest du auf eine methodisch saubere und zukunftssichere Lösung setzen.

Empfohlener Tech-Stack: BoTorch für das gesamte Projekt

Da du einen sequenziellen 3-Schritt-Ansatz planst, solltest du die Architektur in BoTorch wie folgt aufbauen:

- **Für den Multi-Fidelity-GP (Schritte 1, 2 und 3):**

Nutze `botorch.models.gp_regression_fidelity.SingleTaskMultiFidelityGP`. Diesem Modell übergibst du deine Daten inklusive einer zusätzlichen Spalte für die Skala (z. B. kodiert als 0.0 für Micro, 0.5 für Bench, 1.0 für Pilot). Das Modell lernt dann automatisch, wie die billigen Skalen mit der teuren Pilot-Skala korrelieren.

- **Für die Akquisitionsfunktion (Schritte 1, 2 und 3):**

Nutze

`botorch.acquisition.max_value_entropy_search.qMultiFidelityMaxValueEntropySearch` oder `botorch.acquisition.knowledge_gradient.qMultiFidelityKnowledgeGradient`. Diese Funktionen bewerten inhärent die Kosten der Skala, die du über ein Kostenmodell (z. B. eine einfache Funktion, die je nach Skalen-ID 10, 200 oder 2000 zurückgibt) einspeist.

Der Workflow im Code (für jeden deiner 3 Schritte gleich):

1. **Initialisierung:** Erstelle ein paar Startpunkte mit pyDOE2 (für Latin Hypercube Sampling) oder der eingebauten Sobol-Sequenz in BoTorch.
2. **Modellierung:** Füttere die Punkte in den SingleTaskMultiFidelityGP.
3. **Optimierung der Acquisition:** Lasse BoTorch den nächsten Punkt *und* die nächste Skala vorschlagen.
4. **Übergabe an Simulator:** Führe den Versuch im Simulator aus, fahre das Ergebnis ein und wiederhole die Schleife, bis das Budget für den aktuellen Schritt aufgebraucht ist.

Besitzt du bereits grundlegende Erfahrungen mit PyTorch, oder wäre eine Bibliothek mit einer flacheren Lernkurve wie Emukit trotz der älteren Codebasis für dich im Moment leichter zugänglich?

2. Chat (Claude.ai/ Sonnet 5 (medium))

This is my working code for a BO-Loop. I am currently working on refining the termination function/ criteria.

My plan is that for a BO-Loop to terminate two things must be true at the same time.

1. The yield of the found point in the current BO-Loop has to be greater than what has been found before. The structure of this code is, that 3 BO-Loops run after each other. The first one optimizes Temperature. Since this is the first loop the minimal yield shall be extracted from the initialization experiments. The BO-loop may only terminate if the found yield is greater than the single highest found during initialisation. In the following BO-Loops the found Yield must be greater than any yield found before hand.
2. For the last 7 runs the code is only testing basically the same point. If the reaction parameters (T in the first loop, pH in the second and F1, 2, F3 in the third loop). For this a threshold of 0.003 should be implemented in T and pH. For the feed rates there should be a threshold of 0.04 based on the euclidean norm.

It will be your task to check if these termination criteria are correctly employed in the code.

Before you start please summarize in 5 sentences what you understood.

Do nothing else, No confabulation.

+Current code as .txt format

Claude:

I've now reviewed the full code. Here's my understanding in 5 sentences:

The code runs three sequential BO loops (Step 1: Temperature, Step 2: pH, Step 3: F1/F2/F3), and for each loop's early-stopping check, it first computes a yield condition — comparing the current best yield to `avg_init_yield` (average of bench-scale initial samples) in Step 1, or to `highest_prior_yield` (max yield across all prior logged results) in Steps 2/3. If this yield condition holds AND at least `EARLY_STOP_WINDOW` (7) iterations have accumulated, it then checks parameter convergence: for Steps 1/2 via `check_convergence_1d`, which looks at the relative change of the *best-so-far* parameter value across the last 7 iterations against `EARLY_STOP_REL_THRESHOLD` (0.002, i.e., 0.2%); for Step 3 via `check_convergence_nd`, which normalizes F1/F2/F3 to [0,1] using the bounds and checks the Euclidean distance between window-start and window-end

against `EARLY_STOP_DIST_THRESHOLD` (0.04). Convergence in Steps 1/2 additionally triggers a linear extrapolation of the parameter value plus a GP-based validation step (`validate_extrapolated_point`) before accepting it as the final point, falling back to the best observed point if the extrapolation isn't confirmed by the model's posterior mean. Step 3's convergence has no extrapolation and directly uses the best observed point instead. The termination check runs once per iteration inside the loop, after the new data point is added to X/Y, and only breaks out of that step's loop when both the yield condition and the corresponding convergence condition are simultaneously true.

One discrepancy I noticed while reading (flagging per your instructions, not fixing yet): your request specifies a threshold of **0.003** for T/pH, but the code's `EARLY_STOP_REL_THRESHOLD` is set to **0.002**. Let me know if you'd like me to proceed with a full check against your two stated criteria now.

User:

I see a problem with my code. You describe, that the best so far parameter is checked, if it changed over the last 7 trails. I want the code to check if the last 7 tested parameters differ more than the threshold from each other.

Claude:

Good catch — and looking back at the code, there's actually a deeper inconsistency here than just "start vs. end."

What the code actually does today:

```
best_so_far_history.append(x_next.clone())
```

This appends `x_next` — the point **actually tested** this iteration — every single iteration, regardless of whether it improved on anything. So despite its name and despite the docstring of `check_convergence_1d` explicitly saying it checks "*der BISHHER BESTE Parameterwert (nicht der zuletzt vorgeschlagene Kandidat!)*" ("the best-so-far value, not the last proposed candidate!"), the list is really a history of the **last 7 tested points**, not the best-so-far trajectory. The docstring and the actual behavior contradict each other.

The actual bug you're flagging: even though the history is (accidentally, correctly) the last-7-tested-points, `check_convergence_1d` only compares the **first vs. last point of that window**:

```
start_val, end_val = recent[0], recent[-1]
```

```
...
```

```
rel_change = abs(end_val - start_val) / denom
```

So a sequence like [35.0, 40.0, 30.0, 45.0, 25.0, 50.0, 35.01] would pass as "converged" — the endpoints are nearly identical, but the 7 points bounced all over the search space in between. That's clearly not what you want.

The same flaw exists in `check_convergence_nd`, which also only compares `recent[0]` vs `recent[-1]`.

Proposed fix: replace the endpoint comparison with a **full-window spread check** — i.e., the last 7 tested points must all be mutually within the threshold of each other. For 1D this is equivalent to checking $\max(\text{recent}) - \min(\text{recent})$ (relative); for the 3D feed-rate case it means checking the max pairwise normalized Euclidean distance across all 7 choose 2 = 21 pairs in the window, not just the two endpoints.

This also affects the extrapolation step in the 1D case: right now it extrapolates a "slope" from the window — but if the window is genuinely just noisy repeats of the same point (not a trend), a slope-based extrapolation doesn't make much sense. I'd propose using the **mean of the window** as the converged point estimate instead, then still running it through your existing `validate_extrapolated_point` GP check.

Before I implement this, two quick things to confirm:

User:

Q: For the 1D case (T, pH), once convergence is detected across the last 7 points, what should be used as the final converged value? A: Keep slope-based extrapolation, but based on full-window spread check

Q: Also fix the earlier flagged discrepancy: `EARLY_STOP_REL_THRESHOLD` is currently 0.002, but you specified 0.003? A: No, leave at 0.002

Claude:

Here's what I understood:

You've identified that using a single **relative** threshold (`EARLY_STOP_REL_THRESHOLD`, currently 0.002) for both T and pH is numerically problematic, because pH operates on a much smaller absolute scale (~3–9.5, optimum ~8) than T — so 0.002 relative to a pH value of 8 corresponds to an absolute spread of only ~0.016, which is unrealistically tight and essentially prevents the pH loop's convergence check from ever triggering. Your fix is to stop sharing one relative threshold between T and pH, and instead give each parameter its **own dedicated convergence barrier**, presumably calibrated to its own physically meaningful scale rather than a shared relative percentage. This change is scoped only to `check_convergence_1d` (the parameter-spread/convergence check) for the T-loop and pH-loop individually — the F1/F2/F3 (nd) convergence check is not part of this change. You've explicitly stated the **yield criterion** (condition 1 of termination) is to

Group 8

[Hier eingeben]

Dziyana Medvetskaya (243210)

Julian Krüger (235696)

Philipp Johanning (232627)

Nils Guhl (176506)

remain completely untouched. I have not yet made any code changes — awaiting your go-ahead on how exactly you'd like the two barriers defined (e.g., separate absolute thresholds vs. separate relative thresholds per parameter).

[...]